

Tesina

Licenciatura en Estadística

Análisis de comentarios en redes sociales con latent Dirichlet allocation



Facultad de Ciencias Económicas y Estadística

Alumna: Alfonsina Badin

Director: Ignacio Evangelista

2025

Índice

1 Agradecimientos

2 Resumen

3 Introducción

A lo largo de los años, las redes sociales se han convertido en el medio de comunicación más utilizado. Diariamente, los usuarios ingresan a ellas para conversar con otras personas, dar a conocer sus puntos de vista, compartir experiencias, informarse sobre las últimas noticias y entretenerse.

En el año 2020, a raíz de la pandemia mundial por COVID-19, hubo un tipo de contenido que se popularizó con gran velocidad: el *streaming*. El *streaming* es una combinación de radio y televisión, que se transmite en vivo en una plataforma gratuita (YouTube o Twitch) y luego es publicado *On-Demand* para que el usuario pueda verlo fuera de su horario habitual si así lo desea. Por lo general el formato consta de un panel de conductores que charlan de tópicos comunes para la audiencia, generando debates interesantes con los que los usuarios se pueden llegar a identificar. Dentro de cada canal de *streaming* se transmiten distintos programas que tienen un perfil particular y objetivos diferentes ya que su contenido puede variar en entretenimiento, humor, chimentos, noticias, etc.

Su popularidad se debe a que es un contenido gratuito, cercano y que genera una **comunidad** entre los conductores y sus oyentes, quienes no sólo interactúan en vivo a través de un *chat* sino que dejan sus comentarios en otras redes sociales: Instagram, TikTok, YouTube, entre otros.

Al publicar los programas *On-Demand*, los canales de *streaming* acceden a un recurso valioso que refleja la opinión de sus oyentes: los comentarios de YouTube. El análisis de estos textos puede resultar en conclusiones interesantes para los canales ya que permite un entendimiento de los sentimientos que tiene su comunidad con respecto a cada contenido.

Para llevar a cabo este análisis, el presente informe utiliza **latent Dirichlet allocation (LDA)**, una técnica de aprendizaje automático no supervisado que clasifica automáticamente los textos en diferentes categorías o temas según las características del corpus. Esta metodología permite detectar patrones temáticos recurrentes en grandes volúmenes de datos textuales, como los comentarios de YouTube, y extraer información valiosa.

LDA se basa en un modelo probabilístico generativo y es especialmente útil para modelar datos discretos como textos. Es un modelo bayesiano jerárquico de tres niveles, en el cual cada documento (en este caso, cada comentario) se representa como una mezcla de temas, y cada tema se define como una mezcla de palabras. En este sentido, LDA no solo agrupa comentarios en función de temas compartidos, sino que también asigna probabilidades a cada palabra dentro de cada tema, proporcionando una representación explícita y estructurada del contenido del corpus.

Esta capacidad de modelar documentos como combinaciones de múltiples temas lo hace particu-

larmente adecuado para analizar la diversidad temática de los comentarios de YouTube. Por ejemplo, un comentario podría estar relacionado en un 60% con el tema “entretenimiento” y en un 40% con el tema “noticias”, lo que permite identificar la interacción entre los intereses de los usuarios.

Esta tesina tiene por objeto de estudio la técnica latent Dirichlet allocation. Se brinda una introducción al tema, abarcando las definiciones básicas, sus componentes, sus variantes, sus ventajas y limitaciones y sus campos de aplicación. Se incluyen también los conceptos teóricos necesarios para comprender la metodología. Adicionalmente, se presenta la aplicación del modelo en comentarios de redes sociales.

4 Objetivos

4.1 Objetivo general

El objetivo general del presente proyecto es profundizar en el estudio y aplicación del modelo latent Dirichlet allocation (LDA) para la identificación de tópicos o categorías.

4.2 Objetivos específicos

- Comprender las bases del procesamiento de lenguaje natural (NLP por sus siglas en inglés) y las técnicas de representación computacional de texto.
- Mencionar los desafíos y limitaciones que conlleva el ajuste de LDA y propuestas que han surgido para contrarrestar dichos inconvenientes.
- Aplicar LDA en un conjunto comentarios en español (argentino) en la red social YouTube para comprender las opiniones de los oyentes.

5 Metodología

5.1 Machine learning

5.1.1 Aprendizaje supervisado

5.1.2 Aprendizaje no supervisado

5.2 Procesamiento de Lenguaje Natural (NLP)

El texto, tal como lo leemos y comprendemos los humanos, es rico en significado pero carece de una estructura numérica, que es la base del procesamiento en los algoritmos de aprendizaje automático. El procesamiento de lenguaje natural es un área de investigación dentro de la informática y la inteligencia artificial (IA) que se ocupa de procesar el lenguaje humano. Este procesamiento generalmente implica traducir el lenguaje natural en datos (números) que una computadora puede usar para aprender sobre el mundo.

El proceso abarca tareas como la recolección, limpieza, normalización y tokenización de texto, estas últimas dos son las responsables de que la información textual pueda ser estructurada de manera que los modelos la interpreten y analicen eficazmente. En la Figura ?? (?) se pueden observar los pasos que comprende el proceso.

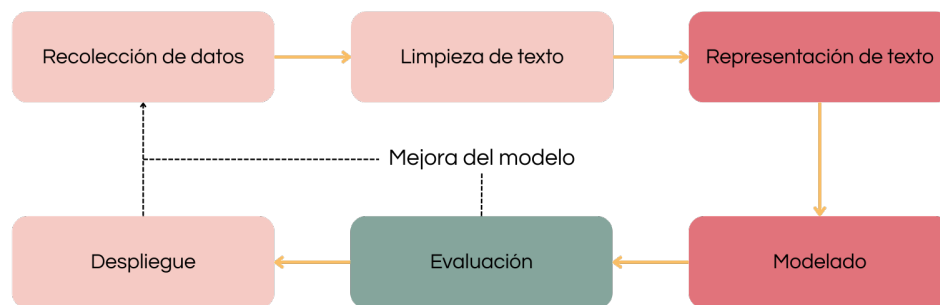


Figura 1: Etapas del NLP

Aunque no sea evidente, el procesamiento de lenguaje natural (NLP) aparece en el día a día de las personas. Tiene una amplia variedad de aplicaciones prácticas, lo que lo convierte en una herramienta esencial en diversos campos, por ejemplo:

- mejoras en motores de búsqueda web,
- corrección ortográfica durante la búsqueda,
- revisión gramatical y ortográfica,
- desarrollo de chatbots y asistentes virtuales,
- generación de índices y tablas de contenido,
- filtrado de spam en correo electrónico,
- personalización en campañas de marketing.

Este amplio rango de usos subraya su importancia como puente entre los lenguajes humanos y las capacidades computacionales.

5.2.1 Recolección de datos

El primer paso esencial en cualquier proyecto de procesamiento de lenguaje natural es la recolección de datos. Esta etapa determina la calidad y relevancia de los resultados que se obtendrán al aplicar técnicas avanzadas como latent Dirichlet allocation (LDA). Existen diferentes enfoques para recopilar datos en NLP, dependiendo de los objetivos del análisis y la disponibilidad de recursos. A continuación, se describen las principales estrategias de recolección de datos:

Uso de conjuntos de datos públicos

Una opción inicial es buscar conjuntos de datos públicos que sean relevantes para la tarea específica. Si el conjunto de datos es adecuado, se puede proceder directamente a construir y evaluar un modelo. Sin embargo, cuando no se encuentra un dataset que cumpla los requisitos, se debe considerar generar datos personalizados.

Obtención de datos de internet

Otra estrategia consiste en identificar fuentes relevantes de datos en internet, como foros de consumidores o plataformas de discusión donde se publiquen consultas. Estos datos pueden ser extraídos automáticamente a través de técnicas de *web scraping* y, posteriormente, etiquetados manualmente por anotadores humanos.

Combinación de fuentes de datos

En la práctica, los conjuntos de datos suelen provenir de fuentes heterogéneas, como *datasets* públicos, datos etiquetados manualmente y datos de internet, al mismo tiempo. Esta combinación es especialmente útil para construir modelos en etapas iniciales, cuando los datos específicos para un escenario particular son limitados.

En esta tesina se optó por hacer uso de una *API* de Google para obtener los datos de entrada para el análisis, este proceso será detallado en la sección ??.

Esta etapa es crucial para garantizar la disponibilidad de datos limpios y útiles, que son la base para las siguientes etapas de procesamiento y modelado. Una vez que se cuenta con los datos recolectados, se procede al siguiente paso: la limpieza y el preprocesamiento de texto.

5.2.2 Limpieza de texto

La extracción y limpieza de texto se refiere al proceso de extraer de los datos de entrada el texto sin procesar, eliminando toda la información no textual, como marcas, metadatos, etc., y convirtiéndolo al formato de codificación requerido. La extracción de texto es un paso estándar en el manejo de datos y, usualmente, no emplea técnicas específicas de NLP. Sin embargo, es un paso importante que tiene implicaciones para todos los demás aspectos. Además, también puede ser la parte más demandante en términos de tiempo dentro de

un proyecto.

Pasos preliminares

Previo a la limpieza del texto, es necesario llevar a cabo una serie de transformaciones básicas que garanticen la homogeneidad del corpus. En primer lugar, se convierte todo el texto a minúsculas con el fin de evitar que palabras idénticas sean tratadas como distintas debido a diferencias en el uso de mayúsculas, por ejemplo, “Video” y “video”. Otro paso fundamental consiste en la eliminación de signos de puntuación, caracteres especiales y espacios innecesarios, que no aportan información semántica al análisis y pueden generar una dispersión artificial en el vocabulario. En esta etapa también se contempla la reducción de espacios múltiples a un único espacio, asegurando así una identificación más precisa de las palabras.

Finalmente, se corrigen patrones de escritura característicos de entornos informales, como la repetición exagerada de letras para enfatizar una emoción (por ejemplo, “amoo” en lugar de “amo” o “encantaa” en lugar de “encanta”). Este tipo de modificaciones, frecuentes en comentarios de redes sociales, pueden aumentar la longitud del vocabulario y dificultar la identificación de términos representativos. La normalización de estas formas al estándar correspondiente resulta clave para mejorar la coherencia interna del corpus y facilitar los pasos posteriores de análisis.

Tokenización

La tokenización es un paso fundamental en el procesamiento del lenguaje natural que tiene como objetivo transformar el texto libre en una secuencia de unidades discretas que puedan ser procesadas computacionalmente. Consiste en segmentar el texto en unidades mínimas llamadas tokens, cuya definición depende de la granularidad aplicada en el proceso:

- A nivel de palabra: cada token corresponde a una palabra. Ejemplo: “Muy bueno el video” → [“Muy”, “bueno”, “el”, “video”].
- A nivel de sub-palabra: cada token es un fragmento de una palabra. Ejemplo: “jugando” → [“jug”, “ando”].
- A nivel de carácter: cada token es un único carácter (letra, signo de puntuación, número, etc.). Ejemplo: “sol” → [“s”, “o”, “l”].
- A nivel de oración: cada token corresponde a una oración completa. Ejemplo: “Hoy llueve. Mañana saldrá el sol.” → [“Hoy llueve.”, “Mañana saldrá el sol.”].

Normalización de Unicode

Al limpiar texto, especialmente que se ha obtenido a través de Web Scrapping, es posible que se encuentren varios caracteres Unicode, incluidos símbolos, emojis y otros caracteres gráficos, algunos ejemplos se muestran

en la Figura ?? (?).



Figura 2: Caracteres Unicode y su representación gráfica

Para interpretar estos símbolos no textuales y caracteres especiales se utiliza la normalización de Unicode, que convierte el texto en alguna forma de representación binaria para ser almacenado en una computadora. Existen varios esquemas de codificación, y la codificación predeterminada puede variar según el sistema operativo.

El siguiente fragmento de código en R ejemplifica la limpieza de emojis en texto. Utilizando expresiones regulares, se eliminan todos los caracteres que no pertenezcan al alfabeto en minúsculas.

```
comentarios <- c(
  "Me encantó este video ", "Lo volvería a ver!!! ",
  "Qué aburrido no me gustó", "Muy bueno!!! "
)
comentarios_limpios <- gsub("[^a-záéíóúñ ]", " ", tolower(comentarios))
comentarios_limpios <- stringr::str_squish(comentarios_limpios)
comentarios_limpios
```

```
## [1] "me encantó este video" "lo volvería a ver"
## [3] "qué aburrido no me gustó" "muy bueno"
```

La salida es un conjunto de comentarios donde los emojis y otros símbolos fueron eliminados, manteniendo únicamente el texto en minúsculas y sin caracteres no deseados.

Eliminación de stopwords

Las *stopwords* son palabras funcionales que, si bien cumplen un rol gramatical importante (como preposiciones, artículos, pronombres...), no aportan contenido semántico relevante al análisis del significado global del texto.

El objetivo de esta etapa es reducir el ruido lingüístico y centrar el análisis en las palabras que efectivamente comunican información. Por ejemplo, en frases como "gracias por el dato" o "esto es una joya", las palabras *por*, *el*, *es*, *una* son típicamente consideradas *stopwords*, mientras que *gracias*, *dato*, *joya* son términos de interés.

Para llevar a cabo esta tarea se utilizará el paquete `tm` (*Text mining*) que cuenta con un listado de *stopwords* (Tabla ??) y funciones como `removeWords` que permite eliminarlas de un texto dado.

Tabla 1: Algunas stopwords del paquete

de	la	que	el	en	y	a	los	del	se
las	por	un	para	con	no	una	su	al	lo

```
library(tm)
prueba <- c(
  "gracias por el dato",
  "esto es una joya"
)
removeWords(prueba, stopwords("es"))
```

```
## [1] "gracias  dato" "  joya"
```

Corrección ortográfica

La corrección de errores en la escritura es un aspecto fundamental del procesamiento del lenguaje natural ya que los textos pueden contener diferentes tipos de fallos que afectan su comprensión y coherencia. Los errores en la redacción pueden clasificarse en tres categorías principales (*Moyotl-Hernández, 2016*):

- Errores ortográficos: ocurren cuando una palabra escrita no existe dentro del idioma.
- Errores gramaticales: Se presentan cuando las palabras utilizadas existen en el idioma, pero no son correctas en el contexto de la oración.
- Errores de estilo: se refieren a palabras redundantes, ambiguas o repetidas que afectan la claridad del texto.

En este sentido, un corrector ortográfico tiene como objetivo identificar palabras mal escritas dentro de un texto y sugerir la opción más adecuada a partir de un conjunto de términos válidos en el idioma.

Corrección ortográfica empleando distancia de Levenshtein-Damerau

La distancia de Levenshtein o distancia de edición es una medida utilizada para calcular la similitud entre palabras, se trata de un conteo de operaciones requeridas para convertir una cadena de caracteres (una palabra) en otra. Las operaciones de edición son:

- Intersección de un caracter: `hogr` $\rightarrow$ `hogar` (agregar ‘a’ entre la ‘h’ y la ‘g’).
- Eliminación de un caracter: `argentina` $\rightarrow$ `argentina` (eliminar la ‘g’).
- Sustitución de un caracter por otro: `numero` $\rightarrow$ `número` (reemplazar la ‘u’ por la ‘ú’).

Hay una generalización de esta medida que considera el intercambio de dos caracteres como una operación: la distancia de Levenshtein-Damerau. En este método, aparece:

- Transposición de caracteres: `paelta` $\rightarrow$ `paleta` (intercambia el lugar de la ‘e’ y la ‘l’).

El método de corrección basado en la distancia de Levenshtein-Damerau se fundamenta en la búsqueda de la palabra más similar dentro de un diccionario para corregir una palabra mal escrita. Este procedimiento implica generar todas las posibles transformaciones de la palabra errónea mediante las operaciones de edición. Luego, cada una de estas transformaciones se compara con las palabras existentes en el diccionario, y aquellas que coincidan se agregan a una lista de sugerencias. Finalmente, la mejor corrección será aquella con la menor distancia a la palabra original.

En la Tabla ?? se muestran todas las palabras generadas a partir de la palabra errónea `burri` con una sola operación de edición.

Tabla 2: Posibles transformaciones de la palabra `burri` con distancia de edición uno

Operación	Palabras generadas
Eliminación	<code>urri</code> , <code>brri</code> , <code>buri</code> , <code>bur</code>
Inserción	<code>aburri</code> , <code>bburri</code> , <code>cburri</code> , ..., <code>zburri</code> , <code>burris</code> , <code>burria</code> , ..., <code>burriz</code>
Sustitución	<code>aurri</code> , <code>burrs</code> , <code>murri</code> , ..., <code>burro</code> , <code>burrq</code> , ..., <code>burrz</code>
Transposición	<code>ubrri</code> , <code>bruri</code> , <code>burir</code>

El desafío radica en seleccionar la cantidad de operaciones admitidas para la corrección y cuál de estas opciones es la más apropiada. En el siguiente código se define un diccionario ficticio para este caso y se calcula la distancia L-D haciendo uso del paquete `stringdist` para cada palabra del mismo. En la Tabla ?? se observan los resultados.

```

# Paquetes
library(stringdist)
library(dplyr)

# 1. Token a corregir
token <- "burri"

# 2. Diccionario ficticio
diccionario <- c("burro", "burrito", "barri", "burla",
                 "perro", "aburris", "aburro", "caballo")

# 3. Calculamos distancia DL
resultados <- tibble::tibble(
  candidato = diccionario,
  distancia = stringdist(token, diccionario, method = "dl")
) %>%
  arrange(distancia, candidato)

```

Tabla 3: Posibles correcciones para la palabra 'burri'

Candidato	Distancia	Candidato	Distancia
barri	1	burla	2
burro	1	burrito	2
aburris	2	perro	3
aburro	2	caballo	6

Si bien es fácil identificar que **perro** o **caballo** no son buenas correcciones, observando el resultado de la distancia L-D se llega a la misma conclusión. Para este conjunto de datos y el ejemplo en particular, hay correcciones mejores en función de este cálculo.

En síntesis, el uso de la distancia de Levenshtein-Damerau constituye una herramienta potente para la corrección ortográfica, ya que permite identificar palabras candidatas a partir de criterios formales de similitud. No obstante, los desafíos principales radican en establecer un umbral de tolerancia adecuado y en gestionar la carga computacional asociada. Una tolerancia demasiado estricta puede dejar sin corregir errores evidentes, mientras que una tolerancia demasiado laxa puede introducir correcciones incorrectas. A su vez, el proceso de comparar un término erróneo contra todas las palabras de un diccionario extenso es computacionalmente intensivo. De igual modo, disponer de un diccionario amplio, representativo y ajustado al dominio del corpus resulta esencial para que las sugerencias tengan sentido lingüístico y contextual. En

conjunto, ambos elementos, son determinantes para perfeccionar los procesos de corrección ortográfica y garantizar resultados confiables en etapas posteriores del análisis de texto.

Lematización

La lematización es un proceso de normalización léxica en NLP cuyo objetivo es reducir las palabras a su forma canónica o “lema”. Al aplicar lematización:

- Se eliminan variaciones flexivas (tiempo verbal, número, género).
- Se conserva el significado léxico de la palabra.
- Se mejora la coherencia del vocabulario al reducir distintas formas a un único término.

El paquete `UDPipe` implementa modelos de *Universal Dependencies* que permiten realizar tokenización, etiquetado gramatical y lematización de textos en múltiples lenguas.

```
library(udpipe)
modelo <- udpipe_download_model(language = "spanish-gsd")
udmodel <- udpipe_load_model(file = modelo$file_model)
texto <- c("Los estudiantes estaban estudiando en la biblioteca.")
anot <- udpipe_annotate(udmodel, x = texto, doc_id = 1)
```

Tabla 4: Salida de `UDPipe`: tokens, lemas y categorías gramaticales

Token	Lema	UPOS
Los	el	DET
estudiantes	estudiante	NOUN
estaban	estar	AUX
estudiando	estudiar	VERB
en	en	ADP
la	el	DET
biblioteca	biblioteca	NOUN
.	.	PUNCT

En la Tabla ?? se observan los resultados de la lematización con una oración simple. El código devuelve palabras separadas, los lemas correspondientes y el tipo de palabra procesada, entre ellas:

- DET: determinante.

- NOUN: sustantivo.
- VERB: verbo.
- ADP: adposición.
- PUNCT: puntuación.

Estos son algunos de los tipos de palabras que el paquete `UDPipe` reconoce. Este etiquetado permite filtrar aquellas categorías con mayor carga semántica (sustantivos, verbos, adjetivos, adverbios y nombres propios) y así disminuir la dimensionalidad de la entrada del modelo.

5.2.3 Representación de texto

La extracción de características relevantes (*feature extraction*) es un paso clave para el procesamiento de texto, ya que incluso el mejor modelo de *machine learning* devuelve resultados pobres si la información de entrada es inadecuada. En este apartado se desarrollarán algunas técnicas para transformar texto en una representación numérica robusta que pueda alimentar un algoritmo correctamente.

Representación vectorial

La representación vectorial de un texto consiste en la transformación de una palabra u oración a un vector de números reales dentro de un espacio vectorial de n dimensiones. Esta transformación se realiza a través de un modelo matemático y el vector resultante sirve para cuantificar las similitudes y diferencias entre dos fragmentos de texto. La manera más común de medir esta comparación es a través de la similitud coseno (o coseno del ángulo entre vectores).

Dados dos vectores, A y B , cada uno con n componentes, la similitud coseno se calcula como:

$$\cos(\theta) = \frac{A \cdot B}{|A||B|}$$

Un resultado de 1 implica similitud total (el ángulo θ es 0), y un resultado de -1 implica diferencia total (el ángulo θ es 180). Además de la similitud coseno, la distancia euclídea es otra métrica frecuente para medir la proximidad o lejanía entre vectores.

One-Hot Encoding Una de las formas más sencillas y fundamentales de representar texto numéricamente es mediante la codificación One-Hot. En este enfoque, cada palabra V del corpus se asocia a un identificador único y se representa como un vector binario de dimensión $|V|$. Este vector está compuesto de todos ceros, excepto en la posición correspondiente a la palabra en el vocabulario, donde se ubica un 1.

Para ilustrar, se considera el siguiente corpus: D1: El perro muere. D2: El gato muere. D3: El perro ladra. D4: El gato maúlla.